

The `get_subtree` Tail on MongoDB Community Edition: A Measured Optimization Study for the PageIndex Retrieval Workload

Junyao Dong

Abstract. A companion benchmark first reported MongoDB’s `get_subtree` P95 on a ten-million-node tree at 2.5s, an order of magnitude above PostgreSQL, DuckDB, and SQLite, and framed it as MongoDB’s one weak spot on the PageIndex retrieval workload. This report set out to turn that finding into an optimization plan and then, because the benchmark harness was already in place, to *measure* the plan rather than argue it from documentation alone. Two results reshaped the conclusion. First, the 2.5s figure does not reproduce: re-measured in isolation and under concurrent load, with the collection resident in cache, `get_subtree` holds a stable P95 of roughly 0.3–0.4s—about $3\times$ PostgreSQL, not the $20\times$ the first run implied. The original tail was an artifact of whole-system contention during the multi-engine run. Second, the intuitive fix does not behave the way documented mechanism predicts. Splitting the large `text` field into its own collection (the Subset Pattern), the move we expected to matter most, leaves the P95 unchanged, because on a host whose cache holds the whole collection the scan is gated by the *number* of per-document fetches, not their size. The one lever that helps is the covering index we had initially demoted: served index-only it removes the fetch and buys $\sim 1.4\times$, though extending it to cover the multi-kilobyte `summary` key inflates it to the size of the collection’s own data. The corrected picture is that `get_subtree` is a sub-second, $\sim 3\times$ -PostgreSQL operation with a narrow optimization space; a larger win needs structural denormalization (Nested Sets, a tree-order `_id`, or a pre-computed subtree document), which the write-once shape of the tree makes viable but which we did not measure. Everything here stays on Community Edition, no engine change.

1 The problem, as first reported

The PageIndex retriever reaches storage through four calls: a point lookup, an expand-children, a content fetch, and `get_subtree`, which pulls a node’s descendants to render the tree view and hand a block to the language model. On the ten-million-node dataset the first three are sub-millisecond on MongoDB and need no work. `get_subtree` was the exception. A large subtree matches about 36,000 nodes, and there the initial multi-engine run posted a MongoDB P50 of 30.9ms and a P95 of 2,478ms, against PostgreSQL at 10.7/123ms, SQLite at 11.8/135ms, and DuckDB at 9.9/47ms (Table 1). The median was within a small factor of the others; the tail read more than an order of magnitude worse, and that tail became the optimization target.

Two things then changed the story. The 2,478ms did not survive a controlled re-measurement (Section 2), and the fixes, once run rather than reasoned about, did not rank the way the documentation suggested (Section 4). This report is the corrected account.

Table 1: `get_subtree` latency as first recorded, large dataset ($\sim 36\text{k}$ -node subtree, ms). The P95 shown here is the figure this report re-examines.

	MongoDB	PostgreSQL	DuckDB	SQLite
P50	30.9	10.7	9.9	11.8
P95	2,478	123	47	135

2 Re-measuring the tail

The 2,478 ms was recorded while the other three engines, the write benchmark, and the concurrency sweep ran on the same host. To separate the query’s own cost from that contention we re-ran `get_subtree` on the ten-million-node tree under controlled conditions: one measuring client, the collection freshly loaded and fully resident, the same 200 seeded subtree paths, optionally with N background processes hammering point lookups at the same `mongod`.

In isolation the P95 lands at $\sim 0.30\text{--}0.42\text{ s}$ (P50 $\sim 30\text{--}40\text{ ms}$) and is stable across repeated runs—an order of magnitude below the first figure. Driving the query while 16, 32, then 64 processes contend for the same server barely moves it (Table 2): P95 rises from 331 to 376 ms, $1.1\times$, never approaching seconds. Throughout, the WiredTiger cache holds the whole collection ($\sim 17\text{ GB}$ resident) and reports zero application-thread evictions, so the tail is neither eviction nor ordinary read contention.

Table 2: `get_subtree` re-measured on the 10M tree (one measuring client, id-only projection, ms) while N background processes drive concurrent point lookups.

Concurrent load N	P50	P95	P99	WT evictions
0	31.4	331	476	0
16	39.4	357	555	0
32	44.1	415	534	0
64	39.0	376	627	0

The corrected magnitude is milder than the first run implied: steady-state, `get_subtree` is $\sim 0.4\text{ s}$ at P95 and $\sim 30\text{--}40\text{ ms}$ at the median, roughly $3\times$ PostgreSQL at *both* (its 123 ms tail and 11 ms median), not the $\sim 20\times$ the original P95 alone suggested. The 2,478 ms was a property of the loaded machine—most likely whole-system resource contention, or a cold-cache first touch before the collection was warm—not of the query in steady state. The mechanism below still holds; its magnitude is sub-second.

3 Root cause: the count of fetches, not their size

A secondary index in MongoDB (here on `path`) is a separate B-tree holding only the indexed value and a record id. To return any other field, WiredTiger must *fetch* the full BSON document and apply the projection to it. A point lookup is one such fetch and is cheap. `get_subtree` is tens of thousands of them. The data is cache-resident—and pages in the WiredTiger cache are held *uncompressed*—so the cost is neither disk I/O nor decompression. It is the per-document work of locating each record, materializing the document, applying the projection, and streaming the result through the cursor, multiplied by subtree size.

The initial hypothesis pinned that per-document cost on the wide leaf `text`: each fetch materializes the whole document, `text` payload and all, only to project it away, so removing `text` should make each fetch cheap. The measurement in the next section refutes that reading. Shrinking each document by $\sim 75\%$ leaves `get_subtree` unchanged, because on this host document *size* never gates the scan—document *count* does. The cost is the $\sim 36k$ separate `FETCH` operations, and each one is dominated by locating the record and threading it through the cursor, not by the number of bytes in it once it is already resident and uncompressed in cache. The consequence for optimization is sharp: only *removing* fetches helps. Shrinking them does not, and neither does changing the query operator.

This is also why the relational engines win the operation specifically: a row store visits each matching row but materializes only the requested column—PostgreSQL does not even decompress the JSONB datum, since `node_id` is a separate column—and a column store reads only the requested column in vectorized batches. They do less work per node, and there are tens of thousands of nodes. The same document model that costs here is what makes MongoDB fast everywhere else: storing a node’s fields together makes “fetch one whole entity” cheap (point and children reads), makes in-place updates cheap (no vacuum debt), and keeps the on-disk form compact. The workload plays to that model on every operation but this one.

4 The optimization plan, measured

Because the harness was already in place, we did not stop at recommending fixes; we ran them on the ten-million-node tree and measured (Table 3). Each stays on Community Edition with no engine change. The results invert the intuitive ranking: the Subset Pattern, the move we expected to lead, does nothing here, and the covering index we had demoted is the only lever that moves the number.

Table 3: `get_subtree` fixes measured on the 10M tree (one client). “view” returns `node_id+title+summary` (what the tree view renders); “id” returns `node_id` only. P50/P95 in ms.

Variant	P50	P95	Result
Baseline, view (<code>text</code> inline)	39.6	419	reference
Subset Pattern (split <code>text</code>), view	39.2	419	no change
Lean covering <code>{path,node_id}</code> , id	19.3	191	$\sim 1.4\times$, covered
Fat covering <code>+title,summary</code> , view	33.5	350	$\sim 1.2\times$; 4.66 GB index
Lean covering + batched <code>\$in</code> , view	137	1314	$\sim 3\times$ slower

4.1 Splitting the text field does nothing (here)

The single most intuitive move—the Subset Pattern, splitting `text` into its own collection keyed by node id so the scanned documents are small—was the leading hypothesis, and on this host it is wrong. It shrinks the collection by $\sim 75\%$ (4.97 $\rightarrow$ 1.25 GB of data) yet leaves `get_subtree` P95 exactly where it was (419 $\rightarrow$ 419 ms for the view, 301 $\rightarrow$ 284 ms for id-only, within run-to-run noise). The reason is the one from the previous section: the cache holds the whole collection either way, so document size never gates the scan and the $\sim 36k$ fetches remain $\sim 36k$ fetches.

The design still costs the hot path nothing—`get_subtree` never needs `text`, and the body is read later and separately by `get_entity`, on the few nodes the model selects—so it remains a

reasonable schema choice on its own merits (a smaller working set, a lighter write path). What it is *not*, on this workload and this host, is a `get_subtree` fix. *Caveat, and it is the important one:* the Subset Pattern’s entire premise is a working set too large for RAM, and this experiment ran with a WiredTiger cache many times the collection size, so it could not have exercised that premise. On a cache-constrained production host, where the collection with `text` spills RAM and the version without it fits, the split could matter a great deal. Our result says only that document *size* is not the lever *when the data is resident*; it does not say the split is worthless in general. This is the one open retest worth running (Section 7).

4.2 A covering index is the only real lever, and a modest one

Removing the `FETCH` is the one thing that helps. A lean `{path,node_id}` index answers the id-only subtree query index-only— `explain` confirms `PROJECTION_COVERED`, `totalDocsExamined: 0`, `keysExamined: 12838`—for $\sim 1.4\times$ (P95 301 $\rightarrow$ 191 ms). This is the fix the earlier hypothesis demoted, on the theory that the multi-kilobyte `summary` key would defeat a covering index; the id-only index carries no `summary` and is not defeated.

The complication is that the tree view needs `title` and `summary`, not just `node_id`. Extending the index to cover them (`{path,node_id,title,summary}`) does work—it is still a fully covered scan, `totalDocsExamined: 0`, and it beats the baseline view (419 $\rightarrow$ 350 ms, $\sim 1.2\times$)—but it inflates the index to 4.66 GB, as large as the collection’s own data, because `summary` really is a multi-kilobyte key and WiredTiger’s index prefix compression deduplicates the shared `path` prefixes but does nothing for the fat `summary` values. So the fat-key concern was real: the covering index helps, but the cheap version ($1.4\times$) covers only the id projection, and covering the actual view projection costs an index the size of the data for a $1.2\times$ gain.

Resolving the view a third way—a covered id scan followed by a batched `$in` to fetch `title` and `summary`—is worse, not better ($\sim 3\times$ slower, P95 1,314 ms). A $\sim 36\text{k-id}$ `$in` overflows the 16 MB command limit, forcing ~ 36 chunked round trips, and each chunk re-fetches the same documents the plain scan would have, with cursor overhead on top. Keep the anchored materialized-path range as the predicate—an unanchored regex cannot form a range and must scan the whole index—and treat cache sizing and `zstd` as irrelevant here: the data is already resident and held uncompressed, so neither touches the per-fetch cost.

4.3 A larger win needs structural denormalization (untested)

The covering index shaves fetches at the margin; the structural fixes remove them wholesale, and are the only route to closing the remaining $\sim 3\times$ to PostgreSQL. All three below are viable because the tree is largely write-once after ingest, so their cost is paid once, at load, not on every read.

Nested Sets. Assign each node a `left/right` number in a depth-first traversal at ingest; a subtree is then a single range query, `find({left:{$gt:p.left}, right:{$lt:p.right}})`, with no recursion and no per-node fetch beyond the range. MongoDB recommends this for static trees and warns it is “inefficient for modifying the tree structure” because inserts and deletes require $O(n)$ renumbering. The workload’s incremental-insert path (reindexing adds nodes after ingest) means the tree is not *strictly* write-once, so this fits a load-then-serve deployment or a batched re-number at reindex time, not a steadily growing tree renumbered per insert.

Tree-order `_id` on a clustered collection. A clustered collection stores documents physically ordered by `_id`, in the same file as the index, so a subtree range scan becomes a sequential read rather

than a series of random fetches—provided tree order (a DFS or nested-set number) is encoded into `_id`. The optimizer will not auto-select the clustered index when a usable secondary index exists, so the query needs a hint.

Pre-computed subtree document. Because the tree is write-once in the common case, the structure-and-summary view of each bounded subtree root can be materialized once at ingest and stored as a single document, turning `get_subtree` into a point read. It must exclude `text` to fit the 16 MiB document limit ($1.5\text{ kB} \times 36\text{ k} \approx 55\text{ MB}$ with `text`; structure-and-summary buckets of a few levels stay well under it). The open cost is storage amplification from overlapping subtrees.

We did not measure any of these. With the baseline already at $\sim 0.4\text{ s}$ on an infrequent operation, the honest open question is whether the schema complexity is worth shaving a sub-second tail—which is a product decision, not a mechanism one. (`$graphLookup` is not a candidate: it re-traverses `parent_id`, still materializes documents, and in 7.0 is capped at 100 MB and ignores `allowDiskUse`.)

5 A newer MongoDB version does not close the gap

Upgrading the engine does not fix this without a schema change. Against this operation—a projection-heavy range scan over ordinary documents—the 8.0–8.3 line offers nothing:

- The 8.0 **Express execution path** is a fast path for point and equality lookups on a single index (`EXPRESS_IXSCAN`, `EXPRESS_CLUSTERED_IXSCAN`): `_id`-style queries with no range, sort, or skip. It does not apply to a materialized-path range scan.
- 8.0 **block (vectorized) processing** is scoped to time-series collections and does not extend to general collections through 8.3.
- The `$graphLookup` **disk-spill fix** (SERVER-23980, lifting the 100 MB cap that ignores `allowDiskUse`) lands only in 8.2.0—reverted from 8.1—and even then only helps the recursive-traversal alternative, which loses to the indexed range scan here anyway.

The document-at-a-time `FETCH` is unchanged across the line, so the remedy is schema and index design, not a version bump.

6 What Community Edition does not put on the table

The feature that would solve this most directly is not available without Atlas or Enterprise. There is no **column-store index** (Atlas-only)—the one feature that would make a projection-heavy scan read only the projected column, the way the relational engines do. But `PageIndex` retrieval never scans the whole corpus, so a column store solves a problem this workload does not have; the schema fixes above address the one it does. Self-managed **Search and Vector Search** became runnable on Community in 8.2 (a separate `mongot` process, public preview, replica set required), but they serve relevance retrieval, not a range-projection scan, so they do not address `get_subtree` either. Auto-scaling, stream processing, tiered storage, and federated queries are Atlas-only; the in-memory engine and auditing are Enterprise-only. *No Atlas- or Enterprise-only feature would solve `get_subtree`*: the operation is bound by the document-at-a-time read model, and the only levers are schema, index, and storage configuration—all present on Community.

7 What remains open

Two things the measurements did not settle. First, the exact origin of the original 2,478 ms: point-lookup contention alone does not reproduce it (Section 2), so the cause was heavier than the concurrency the workload itself generates—whole-system resource pressure during the multi-engine run, or a cold-cache first touch. Second, and more useful to close: the Subset Pattern under a *cache-constrained* host. This study ran with the collection fully resident, which by construction cannot show the split’s benefit; a re-run with `-wiredTigerCacheSizeGB` set below the working set (collection with `text` spilling RAM, without it fitting) would test the premise the pattern actually targets. And the measured payoff of the structural-denormalization options remains unquantified. None of these changes the practical conclusion: `get_subtree` is a sub-second, $\sim 3\times$ -PostgreSQL operation whose one cheap fix—a covering index—has limited upside.

8 Conclusion

MongoDB’s one weakness on the PageIndex workload, the `get_subtree` tail, is real but smaller than first reported. A controlled re-measurement puts its steady-state P95 at ~ 0.4 s, roughly $3\times$ PostgreSQL and $\sim 3\times$ at the median too—not the multi-second tail the first run recorded, which traces to contention rather than to the query. The cost is the *count* of per-node fetches, which is why the intuitive fix fails and the demoted one wins: splitting `text` shrinks each document but not their number and leaves the P95 unchanged on a resident collection, while a covering index removes the fetch outright and is the only cheap lever—worth just $\sim 1.4\times$, and only $\sim 1.2\times$ once it must carry the fat `summary` key. A larger win needs structural denormalization the write-once tree makes viable but that we did not measure. The write, concurrency, and footprint advantages that make MongoDB attractive elsewhere are untouched by any of this. The remaining work is not more analysis—it is the cache-constrained Subset retest and the one structural before/after that would price the larger win.

References

1. Subset Pattern. <https://www.mongodb.com/docs/manual/data-modeling/design-patterns/group-data/subset-pattern/>
2. Model tree structures with Nested Sets. <https://www.mongodb.com/docs/v7.0/tutorial/model-tree-structures-with-nested-sets/>
3. Model tree structures with Materialized Paths. <https://www.mongodb.com/docs/manual/tutorial/model-tree-structures-with-materialized-paths/>
4. Clustered collections. <https://www.mongodb.com/docs/v7.0/core/clustered-collections/>
5. WiredTiger storage engine (cache holds data uncompressed; cache sizing). <https://www.mongodb.com/docs/manual/core/wiredtiger/>
6. Query optimization / covered queries; explain results (covered = IXSCAN with no FETCH). <https://www.mongodb.com/docs/manual/reference/explain-results/>
7. `$graphLookup` memory limit. <https://www.mongodb.com/docs/v7.0/reference/operator/aggregation/graphlookup/>
8. MongoDB 8.0 release notes (Express path; block processing). <https://www.mongodb.com/docs/manual/release-notes/8.0/>
9. SERVER-23980 (`$graphLookup` disk spill, fixed 8.2.0); SERVER-102453 (reverted from 8.1). <https://jira.mongodb.org/browse/SERVER-23980>

10. MongoDB Community Edition feature scope (column-store and analytics are Atlas/Enterprise). <https://www.mongodb.com/products/self-managed/community-edition>
11. Independent PostgreSQL-vs-MongoDB benchmark, off-workload (direction only). <https://www.mdpi.com/2504-2289/10/2/66>